Case Study Analysis

Autonomous Agents in Industry 4.0

Monica Raquel Joya
ITAI 3377: IoT and Edge Computing
Professor Ayyagari
Houston City College

Introduction

Industry 4.0 is the name given to today's ongoing shift in how industries make and deliver their products. It is the fourth wave of industrial change, following earlier revolutions built around steam power, electricity, and digital computers. At its core, Industry 4.0 links three main technologies. Cyber-physical systems tie physical machines directly to software. Artificial intelligence helps those systems learn and make decisions. And networks of connected devices, known as the Internet of Things, let factory equipment exchange data in real time. A key part of this shift is the use of autonomous agents. These are software systems that can sense their surroundings, make their own decisions, and act on their own to reach a goal. In a factory setting, autonomous agents can take the place of machines that only follow fixed instructions. The result is a system that is flexible, can adapt to change, and can improve itself over time. Hjulström's (2022) Bachelor's thesis looks at how autonomous agents trained with reinforcement learning can be used for automated guided vehicles, or AGVs. AGVs are robots that move tools and products around a factory floor. His project tests these ideas in a simulated Industry 4.0 warehouse.

Implementation

Hjulström built a Multi-Agent System in which several AGVs learn to move around a warehouse using reinforcement learning. The specific method he used is called the Double Deep Q-Network, or DDQN. The whole system was coded in Python with help from two libraries called TensorFlow and TF-Agents (Hjulström, 2022).

The test environment was a 10x10 grid with three AGV agents, three task squares where jobs had to be picked up or dropped off, and two charging stations. Each agent could see nine pieces of information at any moment: its own x and y position, the location of its next task, its battery level, and the status of the four squares right next to it. Each of those squares could be empty, a wall, a charging station, or a task square. An agent could take one of four actions each turn: move up, down, left, or right.

The reward system was set up to guide the agents toward good behavior. Completing a task earned +100 points. Crashing into a wall or another agent cost 70 points. Running the battery down to zero cost 100 points. At a charging station, the reward depended on how much battery was left. If the agent showed up with less than one third of its battery, it earned +50 points. If it showed up with more than two thirds still left, it lost 10 points. That rule discouraged agents from recharging when they did not really need to. Every step also carried a small penalty

of 1 point, which pushed the agents to finish tasks quickly instead of wandering (Hjulström, 2022).

Training happened in stages rather than all at once. In the early episodes, an agent only had to complete two tasks before the episode ended. Every 30,000 episodes, the number of required tasks went up by one. After 120,000 episodes, the number capped at six tasks per episode and stayed there for the remaining 80,000 episodes of training. This gradual build-up helped the agents learn faster than if they had been given the full task load from the start. Under the hood, the Q-Network used three hidden layers with 100 neurons each, and the system used a training algorithm called Adam with a very small learning rate of 0.00001. Agents started with no prior knowledge and learned entirely through trial and error across roughly 200,000 training episodes.

Benefits

The results were strong. In the final test run, the three agents completed all 300 assigned tasks, which was 100 per agent, with zero collisions and zero battery depletions. Each agent took an average of 1,392 steps to finish its 100 tasks. That works out to about 2.59 extra steps per task beyond the shortest possible path between locations, which shows that the agents were recharging efficiently without taking big detours. The system reached a stable policy after about 120,000 episodes, which is where the average rewards leveled off. The study showed that reinforcement learning is a workable option for training autonomous agents in real industrial settings. It offers flexibility that hard-coded systems cannot match. The Multi-Agent System setup also let the three agents work in parallel, which increased overall throughput (Hjulström, 2022).

Challenges

Several limits stood out. The biggest is the scope of the testing itself. Everything took place inside a 2D computer model, with no runs on real factory hardware. That gap makes it hard to say how the system would perform in an actual warehouse. A second limit is that the evaluation used a single layout, with one grid size, three agents, and two charging stations. Running the same setup in a larger, busier environment, or one with moving obstacles, could produce very different results. On the development side, setting up TensorFlow and TF-Agents took a lot of time at the start of the project, and training the system was slow. These are known issues with deep reinforcement learning. The rewards graphs also went up and down quite a bit during training, which reflects another known issue. When a reinforcement learning agent learns something new, it can temporarily forget or break something it had already learned. Finally, the reward design itself can be a trap. If the numbers are not tuned well, the agent will chase the rewards instead of the actual goal the designer had in mind. A bigger question the study leaves open is whether the strong results from DDQN would still hold up under real-world conditions that a 2D simulation cannot copy. These include noisy sensor readings, small delays when motors respond to commands, batteries that wear down over time, and human workers moving around on the factory floor. The paper also mentions another method called DDPG as an option, but it never compares DDPG and DDQN side by side on this exact task. A direct comparison would have made a stronger case for why DDQN was the best choice (Hjulström, 2022).

Future Implications

The study points to several interesting next steps. Hjulström (2022) suggests turning the charging stations themselves into intelligent agents. Those smart charging stations could talk to the AGVs about whether they are free or busy, and could help the agents decide where and when to recharge. He also recommends testing reinforcement learning for AGVs in real factories to confirm the simulation results. More broadly, the study shows that autonomous agents can be trained on digital twins, which are virtual copies of a real factory layout. A company could train its agents on a digital twin before the actual factory is even built, so the system is ready to run from day one. As Industry 4.0 keeps growing, the mix of Multi-Agent Systems, deep reinforcement learning, and Cyber-Physical Systems is likely to produce factories that keep improving themselves on the fly, adjusting to changing demands and new layouts without needing to be reprogrammed.

Reflection

I found this case study very insightful. It gave me other angles in the way I think about automation. For example, coming from a background in oil and gas and algorithmic trading, I am used to seeing automation as a set of fixed rules that follow a very specific playbook. What stood out to me about Hjulström's work is that the AGVs did not follow any playbook at all. They learned the best way to move around a warehouse by trying actions, getting feedback, and adjusting over roughly 200,000 training episodes. That felt closer to how a new delivery driver learns the fastest route through a city than how a traditional factory robot is programmed, and it made the idea of industrial reinforcement learning click for me in a way that reading about it in a textbook never did.

The Industry 4.0 connection to our ITAI 3377 course was easy to see. Edge devices, sensors, and Cyber-Physical Systems are the exact building blocks that make this kind of warehouse possible. Reading about how each agent used a nine-piece observation vector to track its position, battery level, and surrounding squares reminded me of the kind of data an ESP32-S3 board could collect at the edge in a real facility. It took ideas I had been studying in the abstract and tied them to a specific real-world use case.

The Challenges section was the most useful part for me. The gap between a 10 by 10 simulation and a real Houston warehouse is big, and the reward design sounds like a trap that is easy to fall into. If the rewards are not tuned well, the agent chases the wrong thing, which is a lesson that transfers directly to algorithmic trading.

Looking ahead, I can see this approach being valuable in Houston's energy corridor and port logistics, where AGVs and automated systems move materials through warehouses and yards every day. A self-learning fleet that adjusts to layout changes on its own could cut downtime that fixed routing simply cannot handle.

Reference

Hjulström, L. (2022). *Autonomous agents in Industry 4.0: A self-optimizing approach for automated guided vehicles in Industry 4.0 environments* [Bachelor's thesis, KTH Royal Institute of Technology]. ITAI 3377: IoT and Edge Computing, Module 09, Houston City College EagleOnline.